

Beyond the Square-Root Exponent for Depth-Three Boolean Circuits

OpenAI

September 23, 2026

Abstract

We construct a language in deterministic polynomial time whose n -bit membership function requires $2^{\omega(\sqrt{n})}$ gates in an unbounded-fan-in OR–AND–OR circuit. The bound holds at every sufficiently large input length and counts all gates, including the bottom layer.

1 Introduction

We study depth-three Boolean OR–AND–OR circuits with arbitrary fan-in and fan-out. Such a circuit is a disjunction of CNFs: conjunctions of clauses, each a disjunction of literals. Classical lower bounds for explicit functions have an exponent proportional to the square root of the input length. We ask whether one polynomial-time language can require more than $2^{A\sqrt{n}}$ gates for every constant $A > 0$, at every sufficiently large length. The word “one” matters: the language and its membership algorithm must be fixed before A is chosen.

Throughout the paper, $S_3(f)$ denotes the minimum *total* number of AND and OR gates in an OR–AND–OR circuit computing f exactly. Every bottom gate and the output gate are counted. Input negations are free. The three layers are successive: bottom gates read literals and optional constants, middle gates read bottom outputs, and the top gate reads middle outputs. Unary gates allow smaller-depth functions in this model.

Theorem 1.1. *There is a language $L \subseteq \{0, 1\}^*$, a deterministic Turing machine deciding L , and fixed positive integers C, a such that the machine runs in time at most $C(n + 1)^a$ on every input of length n . If f_n is the membership function of L on $\{0, 1\}^n$, then*

$$\lim_{n \rightarrow \infty} \frac{\log_2 S_3(f_n)}{\sqrt{n}} = \infty.$$

Equivalently, for every real $A > 0$, there is an integer N_A such that

$$S_3(f_n) > 2^{A\sqrt{n}} \quad \text{for every } n \geq N_A.$$

No uniformity or bottom fan-in restriction is imposed on the circuits.

The square-root exponent. Håstad’s random-restriction and switching method gives the $2^{\Omega(\sqrt{n})}$ depth-three lower bound for parity [6, Theorem 1]. Håstad, Jukna, and Pudlák developed a top-down approach and improved the constants for parity and majority [7, Theorems 3.3 and 3.5]. Paturi, Pudlák, and Zane then obtained the sharp parity bound $\Omega(n^{1/42\sqrt{n}})$ through their satisfiability coding lemma, matching the upper bound within a

constant factor [11, Theorem 6]. Thus parity itself cannot witness the improvement sought here. A further connection between satisfiability algorithms and isolated solutions led Paturi, Pudlák, Saks, and Zane to a code-membership lower bound with leading exponent $(\pi/\sqrt{6})\sqrt{n}$, under their stated code-parameter conditions [10, Theorem 6]. These bounds remain at the constant-times-square-root scale.

Theorem 1.1 makes the ratio of the exponent to $\sqrt{n}$ unbounded. This is the depth-three lower-bound problem discussed by Gurumukhani, Paturi, Pudlák, Saks, and Talebanfard [5, Section 1.1] and still identified as open in [4, Section 1]. It is weaker than a lower bound $2^{n^{1/2+\varepsilon}}$ for a fixed $\varepsilon > 0$, the question raised in [7, Section 5, Problem 2]. The depth remains three throughout.

Bounds for restricted bottom fan-in have a different scale. Paturi, Saks, and Zane constructed logspace-uniform NC^1 functions with $2^{n-o(n)}$ gate lower bounds when bottom fan-in is at most two [12, Corollary 5.4]. Gurumukhani, Paturi, Pudlák, Saks, and Talebanfard connected local enumeration algorithms to majority lower bounds and obtained an exponential bound for bottom fan-in three [5, Proposition 3 and Corollary 5]. In such bounded-width models, counting the middle CNFs can be useful. For arbitrary bottom fan-in, that count alone cannot measure complexity: every Boolean function has a representation by a single CNF. The total gate count in Theorem 1.1 is therefore essential.

The proof strategy. The circuit argument needs a function whose sign has small correlation with every moderately wide CNF. Write g for its sign, equal to $+1$ on accepted inputs and -1 on rejected inputs. Correlation with a CNF indicator H means $|\mathbb{E}gH|$ under uniform inputs. If a middle CNF feeds an exact OR representation of the function, it accepts only positive inputs, so $gH = H$. Small correlation then forces that middle gate to accept few inputs. Summing over the middle gates will contradict the function’s positive acceptance density.

Our main ingredient transfers this correlation problem to CNFs of constant width. Leave each variable live independently with probability p , and assign all other variables uniformly. Lemma 3.1 expresses each restricted width- k CNF as a finite signed combination of width- b CNFs. When $p \leq 1/2$ and pk is bounded, a constant b makes the expected total absolute coefficient mass at most two, independently of the number of clauses. Classical switching estimates also avoid a clause-count parameter, but change the normal form [6, Section 3, Main Lemma]; here the tests remain CNFs. The proof expands by successive simultaneous clause violations. Counting the resulting paths backwards cancels the number of eligible clauses against a reciprocal violation count. This is the source of the clause-count independence.

We construct a language with a data block and parameter blocks. Membership evaluates a polynomial at a linear hash of the data, in the quotient ring specified by another input block. Fixing the parameter bits selects a function of the data bits, which we call a *slice*. For every desired lower-bound constant, probabilistic choices of these bits supply a slice with small correlation against every CNF of a suitable fixed width on most random restrictions. The language’s evaluator simply uses the supplied bits; it never has to find a hard slice or test irreducibility. The input-index construction of Paturi, Saks, and Zane [12, Section 5] uses the same idea of fixing parameters to obtain a hard subfunction.

To prove the slice’s correlation property, we combine the sunflower sparsification method of Impagliazzo, Paturi, and Zane [8, Section 2] with the disjoint refinement of Dell, Husfeldt, Marx, Taslaman, and Wahlén [2, Appendix A of the full version]. Lemma 4.1 gives the needed form: for every fixed width and every $\eta > 0$, the partition has at most $2^{\eta m}$ CNF pieces on m variables, each of the same bounded width and with only linearly many clauses. There are few enough such sparse tests that a moment bound controls them simultaneously. Universal linear hashing and polynomial interpolation provide the limited-independent

signs needed for this bound; Section 5 gives the sources and elementary arguments. Sparse normal forms and random polynomial families also occur in [8, Section 1.2].

The two probability estimates combine without an independence assumption. On restrictions where every narrow test has small correlation, we multiply that bound by the signed representation's coefficient mass and use its expectation. On the exceptional restrictions, the correlation is at most one. This transfers the slice's bound to every width- k CNF. The constant-true CNF is one such test, so the slice is also nearly balanced.

Finally, a circuit with S total gates has at most S middle CNFs, each with at most S clauses, even when bottom gates are shared. Deleting clauses wider than k incurs total error at most $S^2 2^{-k}$. Section 6 chooses k at the square-root scale. The correlations of the truncated CNFs bound the acceptance probabilities of the original middle gates, up to the deletion error. Those probabilities are too small to cover the positive inputs of the hard slice. The constants in the analysis depend on A ; the language and its polynomial-time machine do not.

2 Clauses, correlation, and restrictions

We record the clause and restriction conventions needed for both structural lemmas.

For a finite coordinate set V , the cube $\{0, 1\}^V$ carries the uniform probability measure. Expectations without a specified measure use this measure. A Boolean predicate is identified with its 0/1 indicator. Its associated sign is $+1$ on accepted inputs and -1 otherwise.

A *clause* is an OR of literals, and a *CNF* is an AND of clauses. Tautological and constant-true clauses can be removed. Each remaining nonconstant clause uses distinct variables and contains no constants. Its *scope* is the set of these variables, and its *width* is the size of its scope. It is violated on exactly one assignment to its scope. A constant-false clause has empty scope and width zero. The empty CNF is the constant-true function.

For an integer $b \geq 1$, let $\mathcal{C}_b(V)$ be the set of CNF indicators on $\{0, 1\}^V$ with clause widths at most b . There is no bound on the number of clauses. For $G : \{0, 1\}^V \rightarrow \mathbb{R}$, define

$$\text{corr}_b(G) = \max_{J \in \mathcal{C}_b(V)} \left| \mathbb{E}_{z \in \{0, 1\}^V} G(z) J(z) \right|. \quad (1)$$

The maximum exists: there are finitely many distinct clause indicators, and repetitions do not change a CNF's function. Both constant functions belong to $\mathcal{C}_b(V)$.

A p -random restriction of $[d] = \{1, \dots, d\}$, for $0 < p < 1$, is sampled in two stages. Include each coordinate independently in a live set T with probability p ; then choose an independent uniform assignment ρ to $[d] \setminus T$. For $G : \{0, 1\}^d \rightarrow \mathbb{R}$, write $G_{T, \rho}$ for the function remaining on $\{0, 1\}^T$. Filling the live coordinates with uniform bits produces a uniform point of $\{0, 1\}^d$. Consequently, for any real-valued functions G, H on $\{0, 1\}^d$,

$$\mathbb{E}_x G(x) H(x) = \mathbb{E}_{T, \rho} \mathbb{E}_z G_{T, \rho}(z) H_{T, \rho}(z). \quad (2)$$

We will also use a *cylinder*: prescribing a pattern on $U \subseteq T$ and leaving $T \setminus U$ arbitrary. Its indicator is a conjunction of unit clauses, and its measure on the live cube is $2^{-|U|}$.

3 A restriction bound independent of clause count

A random restriction need not reduce every clause of a large CNF to small width. The following lemma instead expresses the restricted formula as a signed combination of small-width tests. The total absolute coefficient mass has bounded expectation, regardless of the number of clauses. The normalization by counts of violated clauses is what makes this

possible. This is a different conclusion from the classical switching estimate, which also has no clause-count parameter but changes the formula's normal form [6, Section 3, Main Lemma]. Here the tests remain CNFs; the cost is their signed coefficient mass, controlled in expectation.

Lemma 3.1 (Restriction bound). *Let $k \geq 0$ and $b \geq 1$ be integers, let H be a CNF of width at most k on $[d]$, and let $0 < p < 1$. Choose $T \subseteq [d]$ by retaining each coordinate independently with probability p , and choose a uniform assignment ρ outside T . Suppose*

$$\theta = \sum_{\ell=b+1}^k \binom{k}{\ell} \left(\frac{2p}{1-p} \right)^\ell < 1. \quad (3)$$

There are numbers $W(H, T, \rho) \geq 0$, depending only on the formula and the restriction, such that

$$\mathbb{E}_{T, \rho} W(H, T, \rho) \leq \frac{1}{1-\theta} \quad (4)$$

and, simultaneously for every real-valued function G on $\{0, 1\}^T$,

$$|\mathbb{E}_z G(z) H_{T, \rho}(z)| \leq W(H, T, \rho) \text{corr}_b(G). \quad (5)$$

An empty sum in (3) is zero.

Proof. Discard true and tautological clauses from H , and list the remaining clauses as $C_1, \dots, C_N$, retaining their indices even if clauses repeat. Let $E_i \subseteq [d]$ be the scope of C_i . Thus $|E_i| \leq k$ and violation of C_i prescribes one pattern on E_i . A false clause has empty scope and is always violated. All sums and families below are finite; there is no restriction on the finite number N .

Building the finite expansion. Fix T, ρ , and write $v_i(z)$ for the violation indicator of C_i under this restriction. Put $q = \sum_i v_i$. For $U \subseteq T$, define

$$a_U = \sum_{i: |E_i \cap (T \setminus U)| \leq b} v_i, \quad c_U = \sum_{i: E_i \cap (T \setminus U) = \emptyset} v_i. \quad (6)$$

Call an index *easy* at U if it occurs in the first sum, and *hard* at U otherwise. Once the coordinates in U are prescribed, an easy clause has at most b variables still free. The classifications depend only on scopes, not on the values of z . In particular,

$$K = \mathbf{1}[a_\emptyset = 0]$$

is the width-at-most- b CNF obtained by conjoining the initially easy clauses after substituting ρ .

We will correct K to obtain $H_{T, \rho}$. They already agree where $K = 0$ and where $q = 0$. At a point with $K = 1$ and $q > 0$, however, $K - H_{T, \rho} = 1$, and every violated clause is initially hard. Dividing this unit excess among the violated clauses gives

$$1 = \sum_{i \text{ hard at } \emptyset} \frac{v_i}{q}.$$

The next step replaces the common denominator q by counts of easy clauses. This produces further corrections, indexed by paths of successive clause violations.

A path $P = (i_1, \dots, i_j)$ starts with $U_0 = \emptyset$ and satisfies

$$i_h \text{ hard at } U_{h-1}, \quad U_h = U_{h-1} \cup (E_{i_h} \cap T) \quad (1 \leq h \leq j). \quad (7)$$

Write $\mathcal{P}_j(T)$ for the family of these indexed paths. Each step adds at least $b + 1$ coordinates, so their lengths are bounded by $L = \lfloor |T| / (b + 1) \rfloor$. The selected indices are distinct: once an index is selected, its whole live scope belongs to U_h , and it stays easy thereafter. If all selected clauses are violated, their first h indices are counted in c_{U_h} , and hence

$$a_{U_h} \geq c_{U_h} \geq h. \quad (8)$$

Throughout this expansion, a path fraction is defined to be zero when its numerator vanishes. The displayed bound makes every denominator positive on the numerator's support.

Continue to work at a point with $K = 1$ and $q > 0$. For a nonempty path $P = (i_1, \dots, i_j)$, distinguish its remaining correction

$$r_P = \frac{\prod_{h=1}^j v_{i_h}}{q \prod_{h=1}^{j-1} a_{U_h}}$$

from the contribution obtained by replacing the remaining factor q in its denominator with a_{U_j} :

$$t_P = \frac{\prod_{h=1}^j v_{i_h}}{\prod_{h=1}^j a_{U_h}}.$$

Since $q = a_{U_j} + \sum_{i \text{ hard at } U_j} v_i$, this replacement gives

$$r_P = t_P - \sum_{i \text{ hard at } U_j} r_{Pi}, \quad (9)$$

where Pi denotes extension by i . Indeed, on the support of the prefix numerator, the sum over extensions is $t_P(q - a_{U_j})/q$, leaving $t_P a_{U_j}/q = r_P$. If the prefix numerator vanishes, all extension numerators vanish too, so the identity still holds.

Initially $\sum_{P \in \mathcal{P}_1(T)} r_P = 1$. Substituting (9) repeatedly gives, for $0 \leq s \leq L$,

$$0 = 1 + \sum_{j=1}^s (-1)^j \sum_{P \in \mathcal{P}_j(T)} t_P + (-1)^{s+1} \sum_{P \in \mathcal{P}_{s+1}(T)} r_P.$$

At $s = L$ there are no further paths, so the residual sum is empty. Together with the cases $K = 0$ and $q = 0$, this proves the pointwise identity on the whole live cube

$$H_{T,\rho} = K + K \sum_{j=1}^L (-1)^j \sum_{P \in \mathcal{P}_j(T)} \frac{\prod_{h=1}^j v_{i_h}}{\prod_{h=1}^j a_{U_h}}. \quad (10)$$

The construction is finite; no convergence argument is involved.

Each path is a positive weight times a finite convex mixture. A path is *compatible* with ρ if all its selected clauses can be violated simultaneously. Such violations prescribe exactly one pattern $\sigma_P \in \{0, 1\}^{U_j}$, with no conditions on $T \setminus U_j$. Indeed, each selected clause prescribes its unique violating pattern; compatibility says that the prescriptions agree on overlaps and with ρ . Write

$$\mathcal{Z}_P = \{z \in \{0, 1\}^T : z|_{U_j} = \sigma_P\}$$

for this cylinder. Then $\prod_h v_{i_h} = \mathbf{1}_{\mathcal{Z}_P}$. An incompatible path has zero numerator everywhere and can be omitted.

For a compatible path and each h , the function c_{U_h} is constant on $\mathcal{Z}_P$. Every clause in its defining sum uses only coordinates already fixed by ρ and the pattern on U_h . Denote this positive integer constant by c_h , and set

$$\omega(P) = \prod_{h=1}^j \frac{1}{c_h}. \quad (11)$$

On $\mathcal{Z}_P$ we have $a_{U_h} = c_h + w_h$, where

$$w_h = \sum_{i \in I_h} v_i, \quad I_h = \{i : 0 < |E_i \cap (T \setminus U_h)| \leq b\}.$$

Substitute both ρ and the final pattern σ_P into each clause indexed by I_h . The resulting clauses on $T \setminus U_j$ have width at most b , since $U_h \subseteq U_j$. Keep their original indices in this family, including clauses that become true or false and any repetitions. Thus w_h remains exactly the number of violated clauses in this indexed family, even when some violations become constant after the final substitution.

We use the following elementary finite mixture. Given an indexed family of a clauses and an integer $c \geq 1$, uniformly permute the a clause objects and c distinct marker objects. Take the conjunction of the clauses appearing before the first marker. This finite uniform experiment defines a convex mixture of conjunctions of the indexed clauses. If an assignment violates exactly w of the indexed clauses, the sampled conjunction holds precisely when the first object among those w clauses and the c markers is a marker. By symmetry its expectation is

$$\frac{c}{c + w}. \quad (12)$$

The argument includes $a = 0$, $w = 0$, repeated clauses, and constant clauses.

Independently apply this mixture at each h to the reduced clauses indexed by I_h , with $c = c_h$. Form J_P by conjoining all sampled clauses, K , and the unit clauses prescribing σ_P on U_j . Every realization of J_P lies in $\mathcal{C}_b(T)$; here the assumption $b \geq 1$ permits the unit clauses. On the cylinder the independent mixtures have expected product $\prod_h c_h / (c_h + w_h)$, while off the cylinder J_P vanishes. Consequently, as functions on the entire live cube,

$$K \frac{\prod_{h=1}^j v_{i_h}}{\prod_{h=1}^j a_{U_h}} = \omega(P) \mathbb{E}[J_P]. \quad (13)$$

The expectation here is over a finite auxiliary probability space. It does not involve the test function G .

Define $W_0(H, T, \rho) = 1$, and, for $j \geq 1$, let

$$W_j(H, T, \rho) = \sum_{\substack{P \in \mathcal{P}_j(T) \\ P \text{ compatible with } \rho}} \omega(P), \quad W(H, T, \rho) = \sum_{j=0}^{\lfloor d/(b+1) \rfloor} W_j(H, T, \rho). \quad (14)$$

For each fixed restriction, combine (10) and (13), then take the correlation with G and use the triangle inequality. Since K and every realization of every J_P have width at most b , this gives (5). Both W and all the mixtures were chosen without reference to G , so the inequality holds simultaneously for every real-valued G .

Reweighting by a full assignment. It remains to bound $\mathbb{E}W_j$. Generate a uniform full assignment $y \in \{0, 1\}^d$ independently of T , and set $\rho = y|_{[d] \setminus T}$. This gives exactly the required distribution of restrictions. Conditional on T, ρ , the live part of y is uniform. For a compatible

path P , it lies in $\mathcal{Z}_P$ with probability $2^{-|U_j|}$; on that event every selected clause is violated and every c_{U_h} equals c_h . Thus, conditional on each T, ρ , multiplying by $2^{|U_j|}$ exactly undoes the probability of its cylinder. In particular,

$$\mathbb{E}_{T,\rho} W_j(H, T, \rho) = \mathbb{E}_{T,y} \left[\sum_{\substack{P \in \mathcal{P}_j(T) \\ C_{i_1}, \dots, C_{i_j} \text{ violated at } y}} 2^{|U_j|} \prod_{h=1}^j \frac{1}{c_{U_h}(y|T)} \right]. \quad (15)$$

The identity holds separately for every path before summing. It does not assume that compatibility or the denominator counts are independent of the restriction.

Counting paths backwards. Fix y . For $R \subseteq [d]$, let

$$V_R = \{i : C_i \text{ is violated at } y, E_i \cap R = \emptyset\}.$$

For a path counted in (15), put

$$D_h = U_h \setminus U_{h-1}, \quad R_h = T \setminus U_h \quad (0 \leq h \leq j), \quad (16)$$

where D_h is used only for $h \geq 1$. These sets satisfy

$$i_h \in V_{R_h}, \quad D_h \subseteq E_{i_h}, \quad |D_h| > b, \quad R_{h-1} = R_h \cup D_h, \quad c_{U_h}(y|T) = |V_{R_h}|. \quad (17)$$

In particular, all displayed denominator counts are positive. Figure 1 depicts how a forward step removes D_h from the live remainder and a reverse step adds it back.

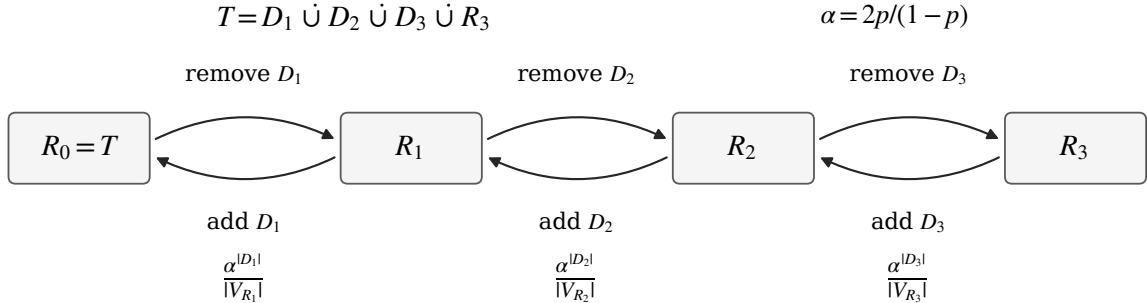


Figure 1: A schematic three-step path for a fixed full assignment y , with clause indices suppressed. Forward steps remove newly prescribed coordinates from the live remainder. Each reverse arrow chooses (i_h, D_h) and adds D_h to R_h . The factor $1/|V_{R_h}|$ averages over eligible violated clause indices at its source state R_h , cancelling their number in the one-step estimate.

The pair consisting of T and its indexed path is determined by the terminal set R_j and the successive reverse choices $(i_j, D_j), \dots, (i_1, D_1)$. Indeed, each choice reconstructs $R_{h-1} = R_h \cup D_h$, recovering $T = R_0$ and every selected index. It is therefore enough to count reverse sequences starting from an arbitrary set R , allowing a choice of any $i \in V_R$ and any $D \subseteq E_i$ with $|D| > b$, followed by the state $R \cup D$. Because $E_i \cap R = \emptyset$, this step always adds D disjointly. All actual reversed paths are included, and allowing additional sequences can only increase a sum of nonnegative weights.

Write

$$\alpha = \frac{2p}{1-p}, \quad \pi(R) = p^{|R|} (1-p)^{d-|R|}.$$

The sets $D_1, \dots, D_j$ partition U_j , and $T = R_j \cup U_j$. The contribution of a given pair (T, P) , including the probability of its live set, therefore factors as

$$\pi(T) 2^{|U_j|} \prod_{h=1}^j \frac{1}{c_{U_h}(y|_T)} = \pi(R_j) \prod_{h=1}^j \frac{\alpha^{|D_h|}}{|V_{R_h}|}. \quad (18)$$

This formula explains the useful direction of counting: at state R , the reciprocal count is exactly $1/|V_R|$, the normalization for averaging over the possible clause indices at that state.

For completeness, define the total weight of s reverse steps from R recursively by $F_0(R) = 1$ and

$$F_s(R) = \begin{cases} \frac{1}{|V_R|} \sum_{i \in V_R} \sum_{\substack{D \subseteq E_i \\ |D| > b}} \alpha^{|D|} F_{s-1}(R \cup D), & V_R \neq \emptyset, \\ 0, & V_R = \emptyset, \end{cases} \quad (s \geq 1). \quad (19)$$

The total weight of the choices at a state with $V_R \neq \emptyset$ is

$$\begin{aligned} \frac{1}{|V_R|} \sum_{i \in V_R} \sum_{\substack{D \subseteq E_i \\ |D| > b}} \alpha^{|D|} &= \frac{1}{|V_R|} \sum_{i \in V_R} \sum_{\ell=b+1}^{|E_i|} \binom{|E_i|}{\ell} \alpha^\ell \\ &\leq \sum_{\ell=b+1}^k \binom{k}{\ell} \alpha^\ell = \theta. \end{aligned} \quad (20)$$

When V_R is empty the total weight is zero. Induction in (19) now yields $F_s(R) \leq \theta^s$ for every R and every fixed y . Notice that (20) averages over the clause indices instead of multiplying by their number. Repeated clauses cause no difficulty: their multiplicities occur in both the sum and $|V_R|$.

Use the injective reverse encoding and (18) to bound the expectation over T inside (15), for this fixed y , by

$$\sum_{R \subseteq [d]} \pi(R) F_j(R) \leq \theta^j \sum_{R \subseteq [d]} \pi(R) = \theta^j.$$

Averaging over y gives $\mathbb{E}W_j \leq \theta^j$. Finally, all terms are nonnegative and $\theta < 1$, so

$$\mathbb{E}W \leq \sum_{j=0}^{\lfloor d/(b+1) \rfloor} \theta^j \leq \frac{1}{1-\theta},$$

as required. Empty formulas, false clauses, and empty live sets are already covered by the construction; if $k \leq b$, there are no nonempty paths and $W = 1$. $\square$

4 A disjoint sparse partition of bounded-width CNFs

We next reduce arbitrary bounded-width CNF tests to tests with only linearly many clauses. The proof adapts the sparsification method of Impagliazzo, Paturi, and Zane [8, Section 2], including its occurrence bounds, insertion accounting, and entropy count of branches. A disjoint refinement is described by Dell, Husfeldt, Marx, Taslaman, and Wahlén [2, Appendix A of the full version]. Here we prove a version with the bookkeeping needed for our partition. Disjointness lets us add the correlations of the pieces without inclusion–exclusion.

Lemma 4.1 (Sparse partition). *For every integer $b \geq 1$ and real $\eta > 0$, there is a positive integer $M = M(b, \eta)$ with the following property. Every CNF indicator H of width at most b on $m \geq 1$ variables can be written pointwise as*

$$H = \sum_{a=1}^q H_a, \quad q \leq 2^{\eta m},$$

where the H_a are CNF indicators of width at most b , each represented by at most Mm clauses, and their satisfying sets are pairwise disjoint. The empty sum is permitted when $H = 0$.

We use the classical sunflower bound of Erdős and Rado [3] to identify a branching step, and include its short proof. A *sunflower* is a collection of distinct sets whose pairwise intersections all equal a fixed set C , called its *core*. The sets obtained by removing C are its *petals*; they are pairwise disjoint.

Lemma 4.2. *Let $s \geq 0$ and $r \geq 2$ be integers. A family of more than $s!(r-1)^s$ distinct sets of cardinality s contains a sunflower with r members.*

Proof. For $s = 0$ there is at most one distinct set, so the hypothesis is impossible. Proceed by induction on s . If the family contains r pairwise disjoint sets, these already form a sunflower with empty core. Otherwise, take a maximal pairwise disjoint subfamily. It has at most $r - 1$ members, and its union U has at most $s(r - 1)$ elements. Maximality implies that every set in the original family meets U . Some element of U therefore belongs to more than

$$\frac{s!(r-1)^s}{s(r-1)} = (s-1)!(r-1)^{s-1}$$

members. Delete this element from all those members. The resulting sets remain distinct and have cardinality $s - 1$. By induction, r of them form a sunflower. Restoring the deleted element gives the required sunflower in the original family. $\square$

Proof of Lemma 4.1. We regard clauses as sets of literals. Discard true and tautological clauses, duplicates, and any clause that contains another clause. If an empty clause remains, use the empty partition and stop. Otherwise the remaining clauses are nonempty and form an antichain under inclusion. The empty family is allowed and represents the constant true function.

The partition will come from a binary branching tree. Fix for now arbitrary integers $r_2, \dots, r_b \geq 2$; we will choose their values after bounding the number of leaves. A sunflower is *eligible at size i* if it consists of r_i size- i clauses and has nonempty core. All set operations below concern literals. In particular, disjoint petals may contain opposite literals of one variable, and a branch may be unsatisfiable.

Branching and disjointness. Each state consists of an active antichain and a separate conjunction of unit clauses, called its *side condition*. Initially the side condition is true. It is never used to select sunflowers or to delete active clauses. Whenever an eligible sunflower exists, choose one of the smallest possible size i , with fixed deterministic tie-breaking, and write its clauses as

$$C \cup P_1, \dots, C \cup P_{r_i}.$$

The core C is nonempty. The petals are nonempty, pairwise disjoint, and all of the same size; their nonemptiness follows from distinctness and equal size of the sunflower members. Make two updates:

- On branch 0, insert the clause C into the active family.

- On branch 1, insert all petal clauses $P_1, \dots, P_{r_i}$ and append to the side condition the requirement that every literal of C be false.

After either insertion, delete all subsumed active clauses to restore the antichain. Every inserted clause has size strictly less than i , so width never increases; the side condition uses only unit clauses. The extra requirement on branch 1 is the disjointness device of [2, Appendix A of the full version]. Keeping it separate ensures that every deletion from the active family is caused by an active insertion, a fact needed in the counting argument below.

Every clause in an insertion batch is new and remains active immediately after the update. Indeed, an inserted clause D is a strict subset of some current clause A . A current clause $B \subseteq D$ would satisfy $B \subsetneq A$, contrary to the antichain property. Within a petal batch, distinct equal-size clauses do not subsume one another either.

Let F be the active CNF indicator before a step, and let F_0, F_1 be the active indicators after its two updates, without side conditions. Subsumption deletion preserves the conjunction, so

$$F_0 = F \mathbf{1}[C \text{ is true}], \quad F_1 = F \prod_{j=1}^{r_i} \mathbf{1}[P_j \text{ is true}].$$

When C is false, the selected clauses in F require every petal to be true. Therefore

$$F = F_0 + \mathbf{1}[C \text{ is false}]F_1, \quad (21)$$

with disjoint supports on the right. Multiplying by the accumulated side condition gives the corresponding disjoint identity for the state. Once we prove termination, iteration of this identity will give a partition into leaf indicators.

Occurrence bounds and insertion budgets. We next bound the number of steps on every finite path prefix. The smallest-size rule bounds how many active clauses inserted in earlier branching steps can contain any one literal. Define

$$e_1 = 1, \quad e_i = (i-1)!(r_i-1)^{i-1} \quad (2 \leq i \leq b).$$

If the current size- i clauses contain no eligible sunflower, each literal occurs in at most e_i of them. For $i \geq 2$, remove a fixed literal from the clauses containing it. More than e_i distinct resulting sets would contain an r_i -member sunflower by Lemma 4.2; restoring the literal gives an eligible sunflower with nonempty core. For $i = 1$, the claim follows because the clauses are distinct.

Call an active clause *added* if it was inserted in a branching step. At every state, for every size u and every literal, we claim

$$\#\{\text{active added size-}u \text{ clauses containing this literal}\} \leq e_u + 1. \quad (22)$$

Initially there are no added clauses. A step can increase this count only by inserting size- u clauses, and such a step processes a size $i > u$. The smallest-size rule then guarantees that there is no eligible size- u sunflower before the insertion. Thus the occurrence bound applies to all current size- u clauses, giving at most e_u occurrences of each literal. The insertion adds at most one further occurrence, because it adds either one core or pairwise disjoint petals. Deletions only decrease the count. This proves the invariant; the initial family itself need not satisfy an occurrence bound.

For any finite path prefix, let N_u count the size- u insertions, counting members of a batch individually. Every deleted added size- u clause was subsumed by a newly inserted strictly smaller clause. Equality is excluded by the freshness of insertions, and side conditions never delete active clauses. Charge each such deletion to one smaller inserted clause that caused it.

A fixed inserted clause D receives at most $e_u + 1$ charges: choose a literal of the nonempty clause D , and apply (22) just before its batch was inserted. Every size- u clause subsumed by D contains that literal. If several batch members could receive a charge, choose only one of them.

At the end of the prefix, at most $2m(e_u + 1)$ added size- u clauses are still active, since each is nonempty and there are only $2m$ literals. Counting deleted and surviving insertions therefore gives

$$N_u \leq (e_u + 1) \left(2m + \sum_{v < u} N_v \right). \quad (23)$$

Define the positive integers

$$D_1 = 4, \quad D_i = (e_i + 1) \left(2 + \sum_{u < i} D_u \right) \quad (2 \leq i \leq b). \quad (24)$$

The same recurrence holds for $i = 1$ with an empty sum. Induction on u in (23) proves

$$N_u \leq D_u m. \quad (25)$$

These bounds hold on every finite prefix, without assuming termination. Every step inserts at least one clause of size less than b , so every path has at most $m \sum_{u < b} D_u$ steps. The binary tree is therefore finite.

For $2 \leq i \leq b$, put $F_i = \sum_{u < i} D_u$. Each step processing size i contributes at least one insertion below size i , and each branch-1 step contributes exactly r_i such insertions. Hence

$$\#\{\text{steps processing size } i\} \leq F_i m, \quad \#\{\text{branch-1 steps processing size } i\} \leq F_i m / r_i. \quad (26)$$

Insertions caused by other processed sizes may also use this budget; including them only enlarges the upper bound.

Counting leaves and choosing thresholds. For each leaf and each $i = 2, \dots, b$, record in order the decisions made at steps processing size i , and pad this binary string with zeros to length $F_i m$. By (26), it has at most $F_i m / r_i$ ones. The tuple of strings determines the leaf. Indeed, distinct leaf paths have a first differing branch: neither can be a proper prefix of the other. Before this difference their states agree, so their selected size and their positions in that size's stream agree. The different bits occur before padding in both streams, so the padded tuples remain different.

Write $h_2(x) = -x \log_2 x - (1-x) \log_2 (1-x)$ for $0 < x < 1$. For integer N and $0 < x \leq 1/2$, the number of binary strings of length N with at most Nx ones is at most $2^{Nh_2(x)}$. A string with $j \leq Nx$ ones has Bernoulli- x probability

$$x^j (1-x)^{N-j} \geq x^{Nx} (1-x)^{N-Nx} = 2^{-Nh_2(x)},$$

because $x/(1-x) \leq 1$. Their total probability is at most one, proving the bound even when Nx is not an integer. Applied to the decision streams, this gives

$$\#\{\text{leaves}\} \leq \prod_{i=2}^b 2^{m F_i h_2(1/r_i)}.$$

We can now choose the thresholds to make this exponent small. The constant F_i depends only on $r_2, \dots, r_{i-1}$, because it involves only D_u with $u < i$. Choose $r_2, r_3, \dots, r_b$ in order, each large enough that

$$F_i h_2(1/r_i) \leq \eta/b, \quad r_i \in \mathbb{N}, \quad r_i \geq 2. \quad (27)$$

This is possible since $h_2(1/r) \rightarrow 0$. An explicit choice is $r_i = 2^{q_i}$, where $q_i \geq 1$ is the least integer satisfying $F_i(q_i + 2)2^{-q_i} \leq \eta/b$: the inequality $-(1-x)\ln(1-x) \leq x$ implies $h_2(2^{-q}) \leq (q + \log_2 e)2^{-q} < (q+2)2^{-q}$. For these choices,

$$\#\{\text{leaves}\} \leq \prod_{i=2}^b 2^{mF_i h_2(1/r_i)} \leq 2^{\eta m(b-1)/b} \leq 2^{\eta m}. \quad (28)$$

Clause counts at the leaves. At a terminal state no eligible sunflower remains. The occurrence bound now applies to the entire active antichain: each literal occurs in at most e_i size- i clauses, so there are at most $2me_i$ such clauses. The side condition contributes at most $2m$ distinct unit clauses after repetitions are removed. Thus the leaf indicator has width at most b and at most Mm clauses, where

$$M = 2 + 2 \sum_{i=1}^b e_i. \quad (29)$$

All constants depend only on b, η . An inconsistent side condition gives an empty piece, which may be discarded. The branching identities (21) give coverage and disjointness, and (28) bounds the number of pieces.

When $b = 1$ there are no branching sizes or streams. The tree has one leaf, its empty product of stream counts is one, and at most $2m$ distinct unit clauses suffice; (29) gives $M = 4$. The constant true function has the empty conjunction representation, and the constant false function can use the empty partition. This covers every $m \geq 1$. $\square$

5 One explicit language and its hard slices

The language will evaluate a polynomial at a linear hash of its data bits. The hash and the polynomial coefficients are themselves input bits. This lets us choose them probabilistically when proving a lower bound, while keeping the membership algorithm fixed. We first define that algorithm on every input, then prove that suitable settings of its non-data bits give small correlation with every bounded-width CNF on most random restrictions. The input-index construction in [12, Section 5] supplies an earlier instance of this evaluator-and-hard-subfunction method. Its lower-bound model has bottom fan-in two; the correlation property proved below is the one needed for arbitrary fixed CNF width.

Definition 5.1 (The language). For an input of length n , put $d = \lfloor n/5 \rfloor$ and reject if $d < 64$. Otherwise define

$$r = \lceil d^{2/3} \rceil, \quad t = \max\{j \in \mathbb{N} \cup \{0\} : j \text{ is even and } j^6 \leq d\}.$$

Reject if fewer than $2d + (t+2)r - 1$ input bits are available. Otherwise parse the input into the following consecutive blocks:

1. d data bits $x = (x_1, \dots, x_d)$;
2. $d + r - 1$ hash bits $u_1, \dots, u_{d+r-1}$;
3. r bits $p_0, \dots, p_{r-1}$, encoding the monic polynomial

$$P(Z) = Z^r + \sum_{i=0}^{r-1} p_i Z^i \in \mathbb{F}_2[Z];$$

4. t blocks of r bits, encoding elements $\beta_0, \dots, \beta_{t-1}$ of the quotient ring $R_P = \mathbb{F}_2[Z]/(P)$, in the basis represented by $1, Z, \dots, Z^{r-1}$.

Unused bits are ignored. Form

$$h_i(x) = \sum_{s=1}^d u_{i+s-1} x_s \pmod{2} \quad (1 \leq i \leq r), \quad h(x) = \sum_{i=1}^r h_i(x) Z^{i-1} \in R_P. \quad (30)$$

Let $\lambda : R_P \rightarrow \mathbb{F}_2$ take the degree-zero coefficient of the unique representative of degree less than r . The input belongs to L exactly when

$$\lambda \left(\sum_{j=0}^{t-1} \beta_j h(x)^j \right) = 0. \quad (31)$$

The matrix in (30) is Hankel (constant on anti-diagonals), or Toeplitz after reversing the data coordinates. Fully random Toeplitz matrices give a standard universal linear hash family [9, pp. 136–137]. We use only the uniform image of each nonzero difference, proved below, not strong pairwise independence of hash values. Once the hash is injective on a restricted cube, polynomial interpolation supplies limited-independent values, as in [1, Section 6, proof of Proposition 6.3].

The rejection rule fixes the language at the small lengths once and for all. For $d \geq 64$, the blocks always fit: they use

$$d + (d + r - 1) + r + tr = 2d + (t + 2)r - 1 \quad (32)$$

bits, and $r \leq 2d^{2/3}$ and $t + 2 \leq 2d^{1/6}$ imply $(t + 2)r \leq 4d^{5/6} \leq 2d$. Thus (32) is at most $4d - 1 \leq n$. We also have $t \geq 2$ and

$$d^{1/6} - 2 < t \leq d^{1/6}, \quad d^{2/3} \leq r < d^{2/3} + 1. \quad (33)$$

Proposition 5.2. *The language L is decided on every input of length n by one deterministic Turing machine in time $C(n + 1)^a$, for some fixed positive integers C, a .*

Proof. After the small-length check, the integer parameters can be computed without real arithmetic: r is the least positive integer with $r^3 \geq d^2$, and t can be found by testing even integers until their sixth powers exceed d . Searching integers up to n and using ordinary integer arithmetic takes polynomial time. Reading and checking the block boundaries does also.

Since P is monic, division by P gives a unique remainder of degree less than r , whether or not P is irreducible. Thus every allowed polynomial block gives well-defined ring operations. Direct multiplication of two coefficient arrays and reduction by P use $O(r^2)$ bit operations: form a polynomial of degree at most $2r - 2$, then successively cancel its highest coefficients by shifted copies of P . Addition uses $O(r)$ bit operations. Formula (30) uses $O(rd)$ bit operations, and Horner evaluation of (31) uses $O(t)$ ring multiplications and additions. The evaluation therefore uses $O(rd + tr^2)$ bit-array operations and polynomial storage.

These are fixed loops on finite bit arrays, with counters of polynomial size. They can be implemented by a fixed Turing machine using direct tape scans, which incur only polynomial overhead per array operation. Together with the initial integer computations this yields a polynomial time bound; increasing its coefficient and exponent to positive integers gives the stated C, a and covers the finitely many small lengths as well. In particular, the machine does not test irreducibility or search for any choice of parameter bits. Neither its instructions nor its running-time constants depend on the proof parameters used below. $\square$

For $d \geq 64$ and fixed non-data bits, write $g : \{0,1\}^d \rightarrow \{-1,1\}$ for the sign of the resulting data function, with $+1$ on accepted inputs. It equals $(-1)^{\lambda(\sum_j \beta_j h(x)^j)}$. Although the evaluator allows every monic modulus, we will select an irreducible polynomial of degree r for the hardness analysis. Such a polynomial exists for every $r \geq 1$; an elementary proof is given in Lemma A.1 in Appendix A. The next proposition is the property of these slices that the circuit argument will use.

Proposition 5.3. *For each fixed integer $b \geq 1$ and real $B > 0$, all sufficiently large lengths n admit fixed settings of the non-data bits such that, with*

$$p = \frac{B}{\sqrt{d}}, \quad m_0 = pd = B\sqrt{d},$$

the resulting sign g satisfies

$$\Pr_{T,\rho} \left[|T| \notin [m_0/2, 2m_0] \text{ or } \text{corr}_b(g_{T,\rho}) > 2^{-|T|/8} \right] \leq 4 \cdot 2^{-m_0/16}. \quad (34)$$

Here T includes each of the d data coordinates independently with probability p , and ρ is an independent uniform assignment outside T . In particular, the correlation bound on a good restriction holds simultaneously for all members of $\mathcal{C}_b(T)$.

Proof. Fix b, B and take d large enough that $0 < p < 1$ and $t \geq 2$. Choose a monic irreducible polynomial P of degree r , and identify R_P with its field F of size 2^r . Initially choose all the hash bits u independently and uniformly, and choose $\beta_0, \dots, \beta_{t-1}$ independently and uniformly in F . We estimate the failure probability jointly over these choices and the restriction; the final step will fix one choice of these input bits.

Injectivity on a restricted cube. For any nonzero $w \in \{0,1\}^d$, the vector $h(w)$ is uniform in $\mathbb{F}_2^r$ over the hash bits. Indeed, let s be the largest coordinate with $w_s = 1$. The i th row in (30) contains u_{i+s-1} with coefficient one, and every other bit it uses has a smaller index. In particular no earlier row uses u_{i+s-1} . The r rows therefore have independent successive pivots, so the linear map $u \mapsto h(w)$ has rank r and takes a uniform input to a uniform output. Equivalently, after choosing the other bits, these pivot bits can be set successively in exactly one way to give any prescribed output. Thus $\Pr_u[h(w) = 0] = 2^{-r}$.

Fix a restriction with $m = |T|$. Its affine data cube has exactly $2^m - 1$ nonzero differences, namely the nonzero vectors supported on T . Since h is linear, two cube points collide exactly when their nonzero difference hashes to zero. A union bound gives

$$\Pr_u[h \text{ is not injective on the restricted cube}] \leq (2^m - 1)2^{-r} \leq 2^{m-r}. \quad (35)$$

The fixed assignment ρ only translates the hash image and has no effect on this estimate.

Limited independence from interpolation. Condition on a hash that is injective on this cube. At any $q \leq t$ distinct cube points, let $a_1, \dots, a_q \in F$ be their distinct hashes. For arbitrary desired values $v_1, \dots, v_q \in F$, the polynomial

$$Q(X) = \sum_{i=1}^q v_i \prod_{\substack{1 \leq j \leq q \\ j \neq i}} \frac{X - a_j}{a_i - a_j}$$

has degree at most $q - 1 < t$ and satisfies $Q(a_i) = v_i$. The denominators are nonzero field elements. Consequently the linear map from $(\beta_0, \dots, \beta_{t-1}) \in F^t$ to the q evaluation values

is surjective. Each fiber is a translate of its kernel and hence has the same size, so uniform coefficients give independent uniform values in $\mathbb{F}^q$. The coordinate map λ is a nonzero $\mathbb{F}_2$ -linear map, with $\lambda(1) = 1$. Its two fibers have equal size, for translation by 1 interchanges them. Applying $(-1)^\lambda$ coordinatewise therefore gives unbiased, t -wise independent signs on the restricted cube.

One bound for every sparse test. Apply Lemma 4.1 with $\eta = 1/8$ and write $M = M(b, 1/8)$ for its positive integer constant. On a cube of dimension $m \geq 1$, the number of possible normalized clauses of width at most b , including the false clause of width zero, is

$$C_m = \sum_{j=0}^{\min(b,m)} 2^j \binom{m}{j} \leq (2m+1)^b.$$

For the inequality, list a clause's literals in a fixed order and pad to length b with a symbol outside the $2m$ literals; this is an injective encoding. A CNF with at most Mm clauses can in turn be encoded by a list of length Mm over these C_m clauses and a padding symbol. In particular the number of such CNF indicators is at most

$$(C_m + 1)^{Mm} \leq 2^{Mm[1+b\log_2(2m+1)]}. \quad (36)$$

This list includes the constant true test (no clauses) and the constant false test (one empty clause).

Fix one such test J , and write $Y = \sum_{z \in \{0,1\}^t} g_{T,\rho}(z) J(z)$. Expectation in the next calculation is over the coefficients β_j , conditional on the injective hash. Expand Y^t as a sum over ordered t -tuples of cube points. If some point occurs exactly once, its sign is independent of all the other distinct signs in the term and has mean zero, so that term's expectation vanishes. Otherwise at most $t/2$ distinct points occur. All such tuples are covered by listing $t/2$ cube points, with repetitions allowed to pad the list, and choosing a position in that list for each of the t factors. There are at most $2^{mt/2} (t/2)^t$ such descriptions. Each term has absolute expectation at most one, and hence

$$\mathbb{E} Y^t \leq t^t 2^{mt/2}. \quad (37)$$

Because t is even, $Y^t = |Y|^t$. Markov's inequality, obtained by bounding $|Y|^t$ below by a threshold on its exceedance event, gives

$$\Pr[|Y| > 2^{3m/4}] \leq t^t 2^{-mt/4}. \quad (38)$$

By (36), the probability that any sparse test has $|\mathbb{E}_z g_{T,\rho}(z) J(z)| > 2^{-m/4}$ is at most

$$2^{Mm[1+b\log_2(2m+1)]+t\log_2 t - mt/4} \leq 2^{-mt/8} \quad (39)$$

whenever

$$\frac{M[1+b\log_2(2m+1)]}{t} + \frac{\log_2 t}{m} \leq \frac{1}{8}. \quad (40)$$

This condition holds uniformly for $m_0/2 \leq m \leq 2m_0$ once d is sufficiently large. Indeed, with b, B, M fixed, (33) bounds the two summands respectively by $O_{b,B,M}(\log d / d^{1/6})$ and $O_B(\log d / \sqrt{d})$, both tending to zero.

If no sparse test fails, any $J \in \mathcal{C}_b(T)$ is, by Lemma 4.1, a sum of at most $2^{m/8}$ indicators from the sparse list. The triangle inequality thus gives

$$|\mathbb{E}_z g_{T,\rho}(z) J(z)| \leq 2^{m/8} 2^{-m/4} = 2^{-m/8}. \quad (41)$$

The sparse event already controls every indicator in the list, so this conclusion holds for every width- b CNF simultaneously. No further probability estimate depends on the choice of J .

Restriction tails and a fixed setting. The variable $m = |T|$ is binomial with mean m_0 . Independence and $1 + v \leq e^v$ give

$$\mathbb{E}2^{-m} = (1 - p/2)^d \leq e^{-m_0/2}, \quad \mathbb{E}2^m = (1 + p)^d \leq e^{m_0}.$$

Markov's inequality in these two estimates yields

$$\Pr[m < m_0/2] \leq 2^{-((\log_2 e - 1)/2)m_0}, \quad (42)$$

$$\Pr[m > 2m_0] \leq 2^{-(2 - \log_2 e)m_0}. \quad (43)$$

Both displayed exponents have coefficient greater than $1/16$. For example, tangent and secant bounds for $1/x$ on $[1, 2]$ give $2/3 < \ln 2 < 3/4$, and therefore $4/3 < \log_2 e < 3/2$. The total probability of leaving the prescribed range is at most $2 \cdot 2^{-m_0/16}$.

For every restriction in that range, (35) and (39) bound the conditional failure probability over the parameter bits by $2^{m-r} + 2^{-mt/8}$. Uniformly in this range,

$$2^{m-r} \leq 2^{-m_0/16}, \quad 2^{-mt/8} \leq 2^{-m_0/16}$$

for all sufficiently large d : the first inequality follows from $r/m_0 \rightarrow \infty$, by taking $r \geq (2 + 1/16)m_0$, and the second from $m \geq m_0/2$ and $t \geq 1$. Adding these two errors and the two restriction tails gives joint failure probability at most $4 \cdot 2^{-m_0/16}$.

For each fixed choice of $(u, \beta_0, \dots, \beta_{t-1})$, let its failure probability be the probability over (T, ρ) of the event in (34). The average of these finitely many numbers is the joint failure probability just bounded. At least one setting therefore has failure probability at most $4 \cdot 2^{-m_0/16}$. Fix that setting, the already selected P , and arbitrary unused bits. This proves the proposition. $\square$

The choices in Proposition 5.3 may depend on b, B and the input length. The quantifiers require only that b, B be fixed before the length tends to infinity. These choices select restrictions of the single language in Definition 5.1; computing membership never requires finding them.

6 The depth-three circuit lower bound

We now combine the two structural lemmas with the hard slices. The order of choices is useful: fix the proposed circuit exponent A , then choose constants B, b , and only then let the input length grow. All choices of parameter bits remain restrictions of the language already defined.

Proof of Theorem 1.1. Proposition 5.2 supplies the single polynomial-time machine. Fix $A > 0$, and put

$$d = \lfloor n/5 \rfloor, \quad s = 3A, \quad B = 64(s + 1),$$

$$p = \frac{B}{\sqrt{d}}, \quad m_0 = B\sqrt{d}, \quad k = \lceil 3(s + 1)\sqrt{d} \rceil.$$

All subsequent assertions hold for every sufficiently large n , with the threshold allowed to depend on A . In particular, $0 < p \leq 1/2$ and $n \leq 9d$, so

$$2^{A\sqrt{n}} \leq 2^{s\sqrt{d}}. \quad (44)$$

Choose a constant target width. Since $k \leq [3(s+1) + 1]\sqrt{d}$ for $d \geq 1$, we have

$$\frac{2pk}{1-p} \leq Q, \quad Q = 4B[3(s+1) + 1].$$

Choose an integer $b \geq 1$, depending only on A , so large that

$$\sum_{\ell > b} \frac{Q^\ell}{\ell!} \leq \frac{1}{2}. \quad (45)$$

Such a choice exists: once $\ell + 1 \geq 2Q$, successive terms have ratio at most $1/2$, and the tails tend to zero. Using $\binom{k}{\ell} \leq k^\ell / \ell!$, the parameter of Lemma 3.1 satisfies

$$\theta = \sum_{\ell=b+1}^k \binom{k}{\ell} \left(\frac{2p}{1-p} \right)^\ell \leq \sum_{\ell > b} \frac{Q^\ell}{\ell!} \leq \frac{1}{2}. \quad (46)$$

This is the only place where the desired circuit exponent affects the target width.

Transfer correlation to width k . Fix a setting of the non-data bits supplied by Proposition 5.3, and let $g : \{0, 1\}^d \rightarrow \{-1, 1\}$ be the sign of that slice. Call a restriction good if it lies outside the failure event in (34). On such a restriction,

$$\text{corr}_b(g_{T,\rho}) \leq 2^{-|T|/8} \leq 2^{-m_0/16}.$$

The exceptional probability is at most $4 \cdot 2^{-m_0/16}$.

For any fixed $H \in \mathcal{C}_k([d])$, Lemma 3.1 gives a nonnegative W_H with $\mathbb{E}W_H \leq 2$ and $|\mathbb{E}_z g_{T,\rho} H_{T,\rho}| \leq W_H \text{corr}_b(g_{T,\rho})$. On an exceptional restriction the same absolute correlation is at most one, since g is a sign and H is an indicator. Equation (2) therefore yields

$$\begin{aligned} |\mathbb{E}_x g(x) H(x)| &\leq 2^{-m_0/16} \mathbb{E}[W_H \mathbf{1}_{\text{good}}] + \Pr[\text{not good}] \\ &\leq 6 \cdot 2^{-m_0/16}. \end{aligned} \quad (47)$$

The good event and W_H need not be independent. Moreover, the setting of the non-data bits was fixed before H was chosen: the good event controls all width- b tests simultaneously, and the restriction lemma applies to each width- k test. Thus (47) holds for every such H for this one slice.

The constant true test is a width- k CNF. Applying (47) to it gives $|\mathbb{E}g| \leq 6 \cdot 2^{-m_0/16}$. The accepted set of the slice consequently has density

$$\Pr[g = 1] = \frac{1 + \mathbb{E}g}{2} \geq \frac{1}{4} \quad (48)$$

for every sufficiently large n .

Remove wide bottom clauses and count every gate. Suppose, for a contradiction, that the n -bit membership function has an OR-AND-OR circuit with $S \leq 2^{A\sqrt{n}}$ total gates. Delete every gate with no directed path to the output, so every remaining middle gate feeds the top OR. Fix its non-data inputs to the setting just chosen. This does not increase its number of gates. There are at most S middle AND gates. Each is the indicator of a CNF H whose clauses are the outputs of its distinct bottom OR gates, and hence it has at most S clauses. A bottom gate shared between several middle gates still supplies at most one clause to each; repeated wires can be removed.

Normalize the clauses after the inputs are fixed. A middle gate with a false clause computes zero and can be omitted. For any remaining middle CNF H , delete every clause of width greater than k , and denote the resulting width- k CNF by H' . Then $H' \geq H$. A normalized clause of width $\ell > k$ is false on a fraction $2^{-\ell} \leq 2^{-k}$ of the cube, so a union bound over its at most S deleted clauses gives

$$\mathbb{E}(H' - H) \leq S2^{-k}. \quad (49)$$

Every input accepted by H is accepted by the top OR, which computes the slice exactly. Hence $gH = H$ pointwise, and

$$\begin{aligned} \mathbb{E}H &= \mathbb{E}(gH) \leq |\mathbb{E}(gH')| + \mathbb{E}(H' - H) \\ &\leq 6 \cdot 2^{-m_0/16} + S2^{-k}. \end{aligned} \quad (50)$$

Here $|g| = 1$ bounds the truncation error, and (47) bounds the first term.

The accepted set is the union of the sets accepted by the middle gates. Summing (50) over at most S of them, and using (44), gives

$$\begin{aligned} \Pr[g = 1] &\leq 6S2^{-m_0/16} + S^22^{-k} \\ &\leq 6 \cdot 2^{-(3s+4)\sqrt{d}} + 2^{-(s+3)\sqrt{d}} \rightarrow 0. \end{aligned} \quad (51)$$

Indeed, $m_0/16 = 4(s+1)\sqrt{d}$ and $k \geq 3(s+1)\sqrt{d}$. This contradicts (48). For the fixed A , every sufficiently large n therefore satisfies $S_3(f_n) > 2^{A\sqrt{n}}$. Since $A > 0$ was arbitrary and the language and its machine are fixed, Theorem 1.1 follows. $\square$

A Irreducible polynomials over $\mathbb{F}_2$

Lemma A.1. *For every integer $r \geq 1$, there is a monic irreducible polynomial of degree r over $\mathbb{F}_2$.*

Proof. First construct an extension field in which $X^{2^r} - X$ splits. Any nonconstant polynomial over a field has an irreducible factor, by choosing a nonconstant divisor of least degree. The quotient by an irreducible polynomial is a field: the Euclidean algorithm expresses 1 as a linear combination of that polynomial and any nonzero residue representative, giving the inverse of the latter. Adjoining a root by this quotient and factoring out the corresponding linear factor can be repeated. Each step reduces the degree still to be split, so finitely many such extensions suffice.

In the resulting field the roots of $X^{2^r} - X$ are distinct, since its formal derivative is $-1 = 1$. There are exactly 2^r roots. Their set F contains 0, 1 and is closed under addition and multiplication: in characteristic two, repeated squaring gives $(a+b)^{2^r} = a^{2^r} + b^{2^r}$, and the analogous product identity is immediate. For a nonzero root a , the identity $(a^{-1})^{2^r} = (a^{2^r})^{-1} = a^{-1}$ gives closure under inversion. Negatives equal the original elements in characteristic two. Thus F is a field with 2^r elements. A basis of F over $\mathbb{F}_2$ has exactly r elements, because a vector space of dimension v over $\mathbb{F}_2$ has 2^v elements.

For $a \in F$, let its monic polynomial of smallest positive degree over $\mathbb{F}_2$ that vanishes at a have degree e . Such a polynomial exists by linear dependence in F , and it is irreducible: a proper factorization would give a smaller vanishing factor in the field F . Polynomial division shows that evaluation identifies $\mathbb{F}_2[a]$ with its quotient field, with basis $1, a, \dots, a^{e-1}$. It therefore has 2^e elements. Regarding F as a vector space over this subfield shows $e \mid r$: if its dimension is v , the products of the two bases give an $\mathbb{F}_2$ -basis of size $ev = r$.

Every element of a field with 2^e elements satisfies $a^{2^e} = a$. For nonzero a , multiplication by a permutes the nonzero elements; taking their product and cancelling gives $a^{2^e-1} = 1$.

Zero satisfies the identity as well. Hence each element of F of degree e is among the at most 2^e roots of $X^{2^e} - X$. Here the usual root bound follows by repeatedly dividing by $X - a$ for distinct roots. If $e < r$ and $e \mid r$, then $e \leq \lfloor r/2 \rfloor$. The number of elements of degree less than r is consequently at most

$$\sum_{e=1}^{\lfloor r/2 \rfloor} 2^e < 2^r,$$

with the sum empty for $r = 1$. Some element of F has degree r , and its monic minimal polynomial is the required irreducible polynomial. $\square$

References

- [1] Noga Alon, László Babai, and Alon Itai. A fast and simple randomized parallel algorithm for the maximal independent set problem. *Journal of Algorithms*, 7(4):567–583, 1986. doi:10.1016/0196-6774(86)90019-2.
- [2] Holger Dell, Thore Husfeldt, Dániel Marx, Nina Taslaman, and Martin Wahlén. Exponential time complexity of the permanent and the Tutte polynomial. *ACM Transactions on Algorithms*, 10(4):21:1–21:32, 2014. doi:10.1145/2635812. Appendix A is cited from the full version, arXiv:1206.1775v1, June 8, 2012.
- [3] Paul Erdős and Richard Rado. Intersection theorems for systems of sets. *Journal of the London Mathematical Society*, s1-35(1):85–90, 1960. doi:10.1112/jlms/s1-35.1.85.
- [4] Mohit Gurumukhani, Daniel Kleber, Ramamohan Paturi, Christopher Rosin, and Navid Talebanfard. Optimal depth-three circuits for inner product. In *41st Computational Complexity Conference (CCC 2026)*, volume 383 of *Leibniz International Proceedings in Informatics*, pages 27:1–27:25. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2026. doi:10.4230/LIPIcs.CCC.2026.27.
- [5] Mohit Gurumukhani, Ramamohan Paturi, Pavel Pudlák, Michael Saks, and Navid Talebanfard. Local enumeration and majority lower bounds. In *39th Computational Complexity Conference (CCC 2024)*, volume 300 of *Leibniz International Proceedings in Informatics*, pages 17:1–17:25. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2024. doi:10.4230/LIPIcs.CCC.2024.17.
- [6] Johan Håstad. Almost optimal lower bounds for small depth circuits. In Silvio Micali, editor, *Randomness and Computation*, volume 5 of *Advances in Computing Research*, pages 143–170. JAI Press, 1989. Author manuscript.
- [7] Johan Håstad, Stasys Jukna, and Pavel Pudlák. Top-down lower bounds for depth-three circuits. *Computational Complexity*, 5(2):99–112, 1995. doi:10.1007/BF01268140.
- [8] Russell Impagliazzo, Ramamohan Paturi, and Francis Zane. Which problems have strongly exponential complexity? *Journal of Computer and System Sciences*, 63(4):512–530, 2001. doi:10.1006/jcss.2001.1774.
- [9] Hugo Krawczyk. LFSR-based hashing and authentication. In *Advances in Cryptology—CRYPTO ’94*, volume 839 of *Lecture Notes in Computer Science*, pages 129–139. Springer, 1994. doi:10.1007/3-540-48658-5_15.
- [10] Ramamohan Paturi, Pavel Pudlák, Michael E. Saks, and Francis Zane. An improved exponential-time algorithm for k -SAT. *Journal of the ACM*, 52(3):337–364, 2005. doi:10.1145/1066100.1066101.

- [11] Ramamohan Paturi, Pavel Pudlák, and Francis Zane. Satisfiability coding lemma. *Chicago Journal of Theoretical Computer Science*, Article 11, 1999. [Journal version](#).
- [12] Ramamohan Paturi, Michael E. Saks, and Francis Zane. Exponential lower bounds for depth three Boolean circuits. *Computational Complexity*, 9(1):1–15, 2000. [doi:10.1007/PL00001598](#).